

Max-Heap

1. Conceito

Uma Max-Heap é uma [árvore binária](#) onde o valor de cada nó é sempre maior ou igual ao valor de seus filhos. Isso significa que a raiz da árvore sempre contém o maior valor. Apesar de ser uma [árvore binária](#), não é uma árvore binária de busca, pois sua estrutura prioriza apenas a propriedade de máximo.

1.1. Características

- Sempre uma [árvore binária](#) completa, com exceção do último nível, que pode estar incompleto.
- O valor de cada nó é maior ou igual ao valor de seus filhos.
- A raiz contém o maior elemento da estrutura.

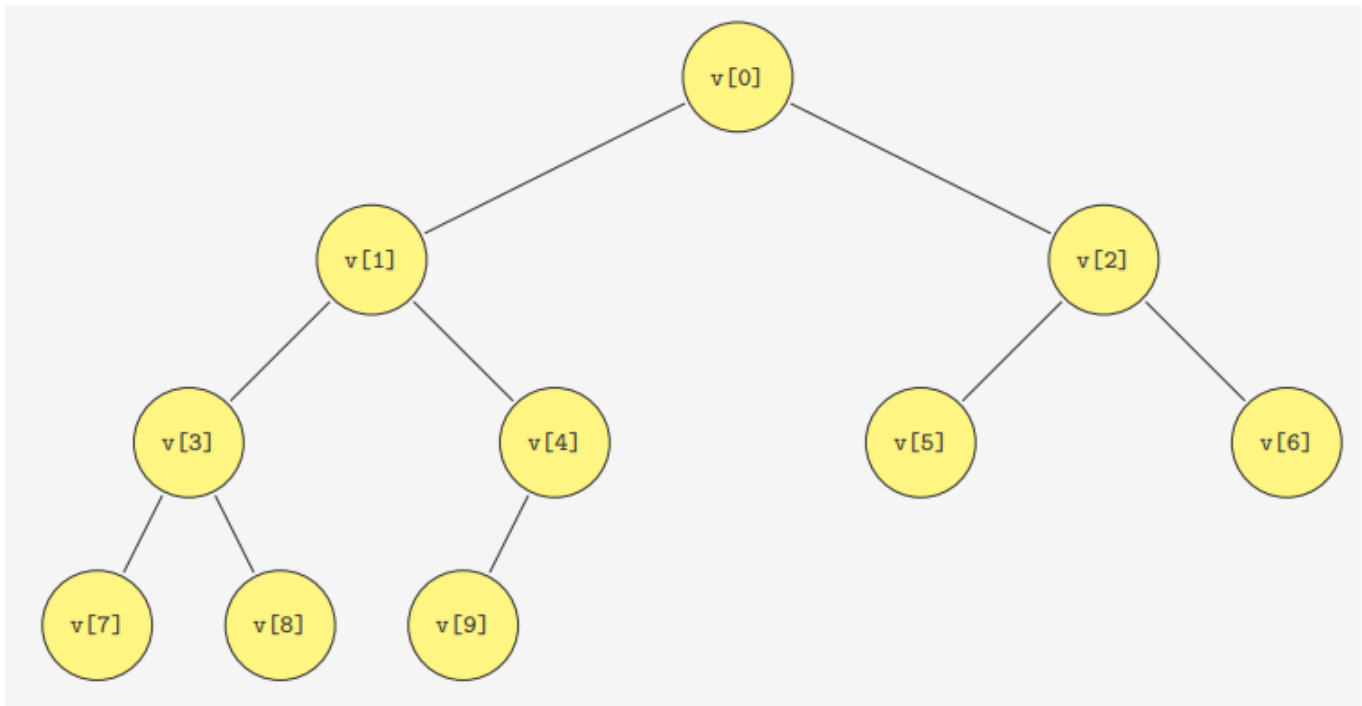


Figura 1: Representação de uma Max-Heap como um vetor.

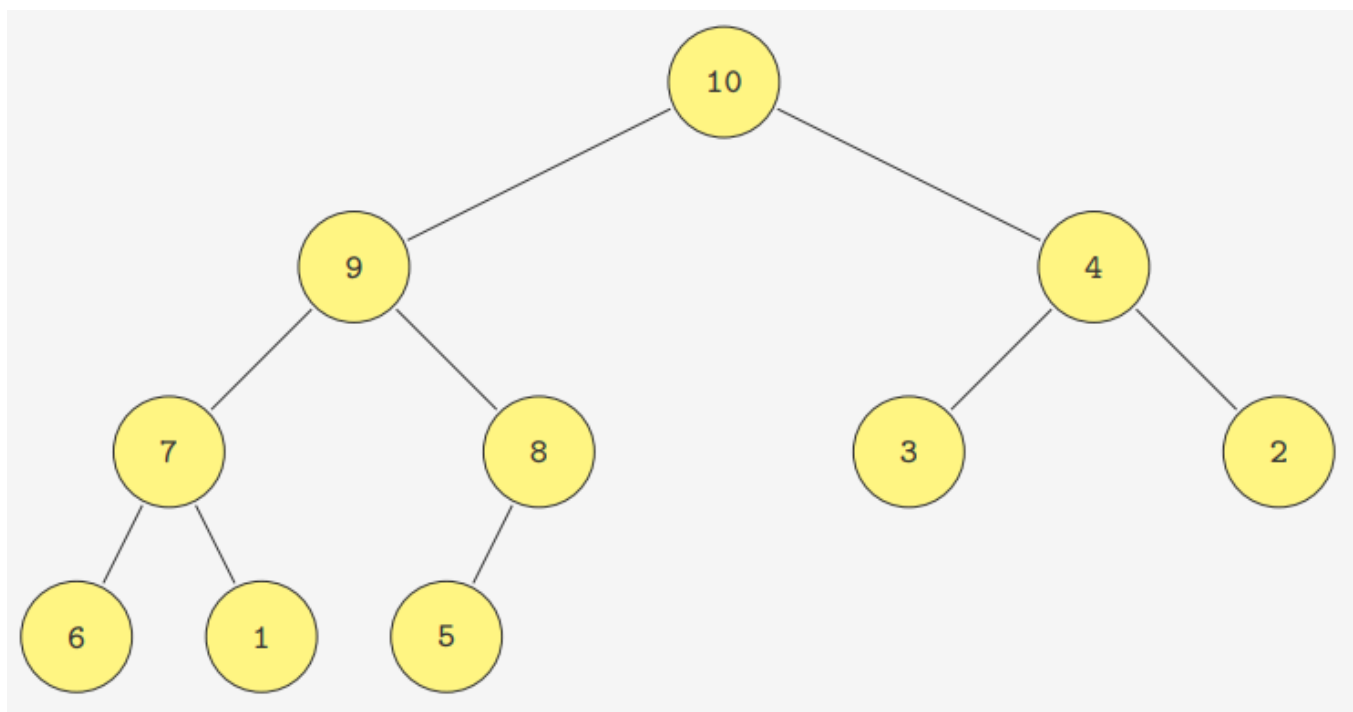


Figura 2: Exemplo visual de uma Max-Heap.

1.2. Relação entre índices no vetor

Dado $v[i]$, podemos determinar:

- Filho esquerdo:
 - $v[(2 * i) + 1]$
- Filho direito:
 - $v[(2 * i) + 2]$
- Pai:
 - $v[(i - 1) / 2]$

1.2. Código criar heap

```
#define pai(i) ((i - 1) / 2)
#define f_esq(i) (2*i+1)
#define f_dir(i) (2*i+2)

typedef struct {
    int *v; // Vetor que armazena os elementos
    int n; // Número atual de elementos na heap
    int tam; // Capacidade máxima da heap
} heap;

heap *criaHeap(int tam) {
```

```
heap *h = malloc(sizeof(heap));  
h->v = malloc(tam * sizeof(int));  
h->n = 0;  
h->tam = tam;  
return h;  
}
```

Explicação:

- `v` : Vetor onde os elementos serão armazenados.
 - `n` : Número atual de elementos na heap, iniciado em 0.
 - `tam` : Capacidade total da heap.
-

2. Inserindo no Heap

A inserção de um novo elemento segue o princípio de subida na heap. O processo é o seguinte:

1. Adicionamos o novo elemento na última posição do vetor.
2. Comparamos o elemento com seu pai.
3. Se for maior que o pai, trocamos os dois.
4. Repetimos o processo até que a prioridade da heap seja restaurada (ou seja, até que o novo elemento não precisa mais subir).

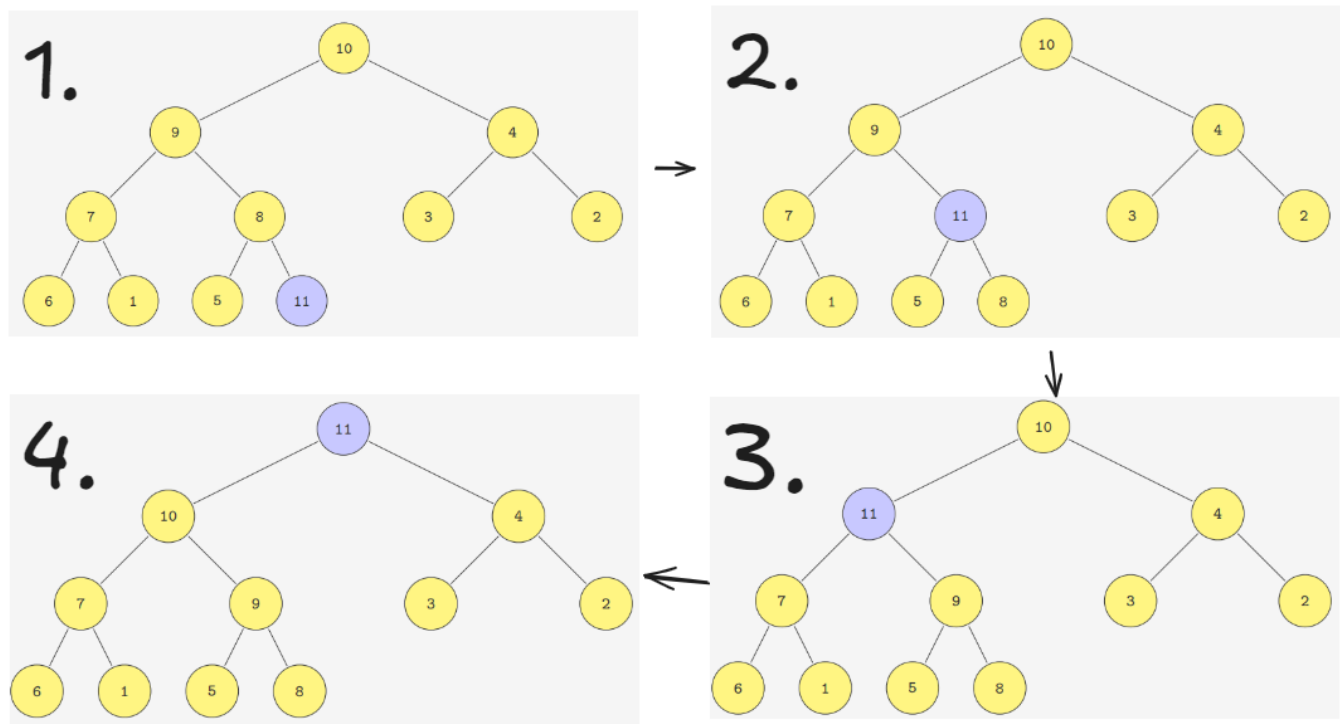


Figura 3: Ilustração da inserção em uma Max-Heap, mostrando o processo de subida do novo elemento.

2.1. Código de inserir

```
void insere(heap *h, int valor){
    if(h->n >= h->tam){
        h->v = realloc(h->v, 2*h->tam * sizeof(int));
        h->tam *= 2;
    }

    h->v[h->n] = valor;
    h->n++;

    // Subindo na heap enquanto necessário
    while (i > 0 && h->v[pai(i)] < h->v[i]) {
        // Troca com o pai
        int temp = h->v[i];
        h->v[i] = h->v[pai(i)];
        h->v[pai(i)] = temp;

        i = pai(i); // Atualiza o índice para continuar subindo
    }
}
```

Explicação:

- Se a heap estiver cheia, **dobramos** sua capacidade.
- O elemento é adicionado na última posição.
- Ele é comparado com seu pai e trocado, se necessário.

2.2. Complexidade

- No pior caso, um elemento pode subir até a raiz.
- Como a heap é uma [árvore binária completa](#), a altura é $O(\log n)$.

3. Remoção no Heap

A remoção na Max-Heap segue um processo de descida para restaurar a propriedade da heap. O elemento removido sempre será a raiz (o maior valor). O processo é o seguinte:

1. A raiz é substituída pelo último elemento.
2. O tamanho da heap é reduzido.
3. O novo valor na raiz é comparado com seus filhos.
4. Se for menor que um dos filhos, troca-se com o maior deles.
5. O processo se repete até que a propriedade da heap seja restaurada.

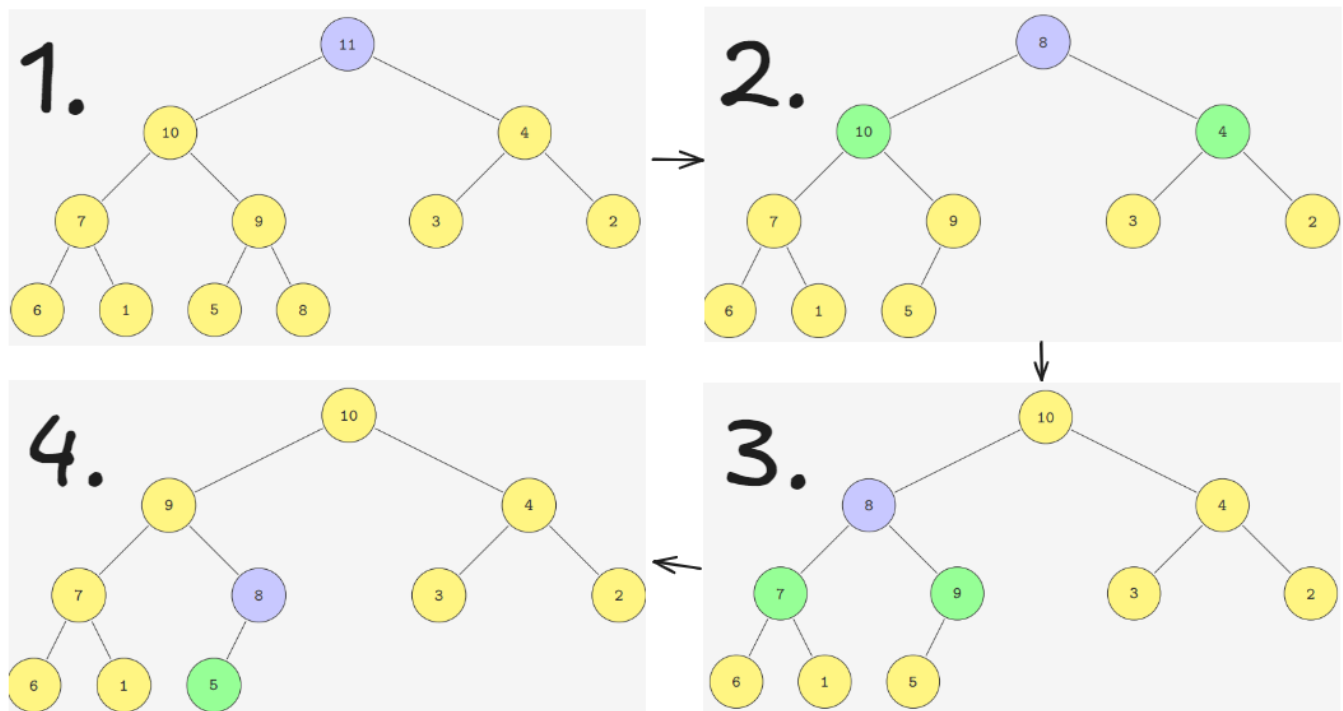


Figura 4: Demonstração de uma remoção em uma Max-Heap, mostrando o processo de descida do novo elemento.

3.1 Código de Remoção

```
int removeMax(heap *h) {
    if (h->n == 0) return -1; // Heap vazia

    int max = h->v[0]; // O maior elemento está na raiz
    h->v[0] = h->v[h->n - 1]; // Substituímos a raiz pelo último elemento
    h->n--; // Reduzimos o tamanho da heap

    int i = 0;

    while ((f_esq(i) < h->n && h->v[i] < h->v[f_esq(i)]) ||
           (f_dir(i) < h->n && h->v[i] < h->v[f_dir(i)])) {
        int maior = f_esq(i); // Assume que o filho esquerdo é o maior

        // Se o filho direito existe e é maior, atualiza o índice do maior
        if (f_dir(i) < h->n && h->v[f_dir(i)] > h->v[maior])
            maior = f_dir(i);

        // Troca o pai com o maior filho
        int temp = h->v[i];
        h->v[i] = h->v[maior];
        h->v[maior] = temp;

        i = maior; // Atualiza o índice para continuar descendo
    }

    return max; // Retorna o valor removido
}
```

Explicação:

- O **maior elemento** está na raiz e é removido
- O **último elemento** é movido para a raiz e desce conforme necessário.

3.2. Complexidade

- A altura de uma Max-Heap é $O(\log n)$.
- No **pior caso**, o elemento trocado desce até a folha.
- Como cada nível da árvore tem **no máximo dois filhos**, o número de trocas é proporcional à altura.

Portanto, a complexidade da remoção também é $O(\log n)$.

